

Detecção de Objetos - Modelo YOLO

Alan Fachini 193398

Felipe Carisio 251001

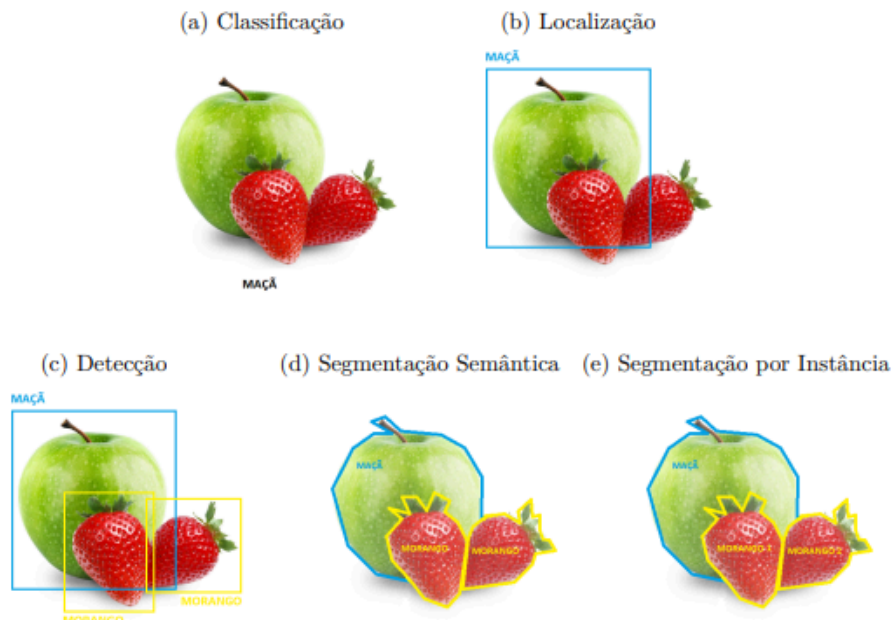
Luís Roberto Marques Júnior 173216

Motivação

A visão computacional é uma área da ciência que, através de ferramentas computacionais, tenta introduzir a capacidade de interpretar imagens para sistemas e computadores; em outras palavras, a visão computacional visa tornar possível que computadores possam “ver”. De formas cada vez mais rebuscadas, espera-se que computadores possam receber imagens, vídeos e extrair dados, reconhecer padrões, fazer a classificação e reconhecimentos dessas imagens e até mesmo tomar decisões através disso.

Portanto, o maior objetivo da visão computacional em si é, de fato, a detecção de objetos. Existem diversas formas de abordar esse problema, mas o padrão mais comum segue o seguinte passo-a-passo: (i) classificação, em que uma imagem é classificada pelo objeto mais relevante presente; (ii) localização, em que se localiza um único objeto em uma imagem; (iii) detecção, localiza e classifica vários objetos em uma imagem; (iv) segmentação semântica, associa cada pixel de uma imagem a uma classe de objeto; (v) segmentação por instância, associa cada pixel de uma imagem a uma instância de objeto.

Figura 1 – Tipos de identificação de objetos



Fonte: DETECÇÃO DE OBJETOS: ESTUDO E APLICAÇÃO DA ARQUITETURA R-CNN (Yan Victor Ribeiro Marim, UFES)

Para o trabalho proposto em questão, trataremos somente até a detecção de imagens, o item c.

Detecção de Objetos

Os avanços na área de visão computacional para detecção de imagens se mostram cada vez mais promissores e requisitados. Conforme a sociedade expande, o volume de atividades a serem monitoradas e a escala dos problemas se tornam cada vez maiores, assim, esperamos que, com maior poder computacional, possamos encontrar técnicas mais efetivas para automatizar um grande volume de funções e melhorar as condições materiais.

Essas novas tecnologias possuem aplicabilidade num amplo espectro de atividades, desde a indústria e o mercado, como Reconhecimento Facial, Sistemas de Vigilância e Veículos autônomos à áreas da Saúde e Bem-Estar como Leitura de Imagens Médicas e Assistência para Deficientes Visuais.

Os primeiros relatos para as tentativas de detecção de objetos de forma regular e computacional datam de 1960, junto com os principais booms tecnológicos envolvendo sistemas computacionais - a própria Pesquisa Operacional tem um boom considerável um pouco antes. Inicialmente, de forma muito rudimentar, eram usadas descrições manuais e configurações geométricas para a identificação de objetos. Como o poder computacional na época ainda era ínfimo, essa abordagem era muito limitada e particular para cada caso, além de bastante ineficiente devido a especificidades das imagens, como oclusão e iluminação.

O primeiro grande avanço nessa área veio na década de 90, com o avanço do Aprendizado de Máquina e a crescente no poder computacional. Com isso a abordagem por descrições manuais caiu em desuso e foi substituída pelo uso de características - um conceito interessante anterior às classes. Junto com tudo isso, a implementação do uso de probabilidades e estatística nas predições elevou em muito as capacidades de reconhecimento visual. O padrão de janelas deslizantes - citado em aula, por exemplo - passou a ser utilizado.

Por fim, o avanço mais relevante veio através das redes convolucionais, ou CNNs. Não mais presos por limitações de poder computacional, redes neurais mais complexas como as CNNs puderam ser implementadas e, particularmente, com grande capacidade de abstração para objetos e detalhes de imagens. Especificamente existem três modelos muito usados para a detecção de imagens: YOLO, R-CNN (*Regions with CNN features*) e SSD (*Single Shot Multibox Detector*). Daqui em diante focaremos somente no modelo YOLO, foco desse trabalho.

O modelo - YOLO

O **YOLO** (*You Only Look Once*) é um algoritmo de detecção de objetos que se diferencia dos modelos anteriores ao tratar a detecção como um problema de regressão, em

vez de usar métodos tradicionais de proposta de região e classificação em múltiplas etapas. Ele se destaca por ser capaz de trabalhar em tempo real, graças à sua arquitetura eficiente, que permite que a imagem seja processada em uma única passagem pela rede. Além disso, o YOLO realiza o treinamento de ponta a ponta, ou seja, toda a rede é treinada simultaneamente, sem dividir o processo em etapas separadas, como em outras abordagens. Essa eficiência faz do YOLO uma solução poderosa e rápida para a detecção de objetos.

Contrário aos métodos anteriores, como a R-CNN, que dividiam o problema em rodar várias janelas detectando objetos, para só então classificar esses objetos e por fim eliminar as duplicatas, o YOLO analisa a imagem somente uma vez. Como a análise é feita uma única vez, o modelo é treinado com imagens inteiras, permitindo encontrar padrões contextuais, diminuindo o número de identificações incorretas no background em comparação com a R-CNN, mesmo nas suas versões mais otimizadas, como a Fast R-CNN. Um contraponto ao YOLO é que, uma vez que o objeto é analisado como um todo, o modelo aprende as representações gerais deste objeto, mas como a informação adquirida não é especializada, o modelo, embora seja bom em classificar os objetos, tem certa dificuldade em localizá-los corretamente.

Figura 2 - Comparativo entre modelos R-CNN e YOLO para detecção de objetos

Comparação Geral: R-CNN vs YOLO		
Aspecto	R-CNN	YOLO
Estratégia de Detecção	Propostas de região + CNN	Regressão direta (previsão de bounding box e classe)
Etapas	Múltiplas (propostas de região, CNN para cada região, SVM para classificação)	Uma única etapa (detecção direta)
Velocidade	Lenta (cada região é processada individualmente)	Muito rápida (detecção em tempo real)
Precisão	Alta, especialmente para pequenos objetos	Boa, mas menos precisa para pequenos objetos
Complexidade Computacional	Alta (cada proposta de região é processada separadamente)	Baixa (uma única passagem pela imagem)
Invariância a Escala	Tratada explicitamente pelas propostas de região	Tratada implicitamente pela grade
Aplicações	Adequado para imagens estáticas e tarefas onde a velocidade não é um fator crítico	Ideal para aplicações de tempo real (como vídeos)

Fonte: OpenAI. (2024). Imagem gerada pelo Chat GPT [IA].

Tudo isso está atrelado diretamente a primeira versão do YOLO, que foi produzida e divulgada pelo seu autor, Joseph Redmon, em 2015 e revisada em 2016. Desde então diversas novas versões foram introduzidas ao modelo, tendo melhoras consideráveis. Atualmente é possível encontrar até a sua versão 10 de forma estável, e a versão 11 no beta. Cogitamos utilizar a mais nova (.v10), porém o grande diferencial dela é justamente a eliminação da

função *Non Max Suppression* - que é um dos focos desse trabalho; com isso em mente e por outras casualidades, usaremos a versão 3 (**YOLO .v3**) que, relativo a sua primeira versão, tiveram melhoras como clusterização de dimensões, normalização de lotes de dados, as *anchor boxes* e *pooling* piramidal.

Arquitetura do YOLO

Foi feita uma contextualização histórica sobre a detecção de objetos até o presente momento, além de citar os modelos mais utilizados e, por fim, a explicação do porque o YOLO foi escolhido, e quais são seus principais pontos. Agora, de uma forma sucinta, a arquitetura de funcionamento do YOLO será, de fato, apresentada.

Ela será dividida em 7 processos: a Entrada, as Camadas Convolucionais, as Camadas de Pooling, as Camadas totalmente conectadas, o Grid de Imagem, as *Bounding Boxes* e classes e, por fim, a Saída.

Entrada (Input)

- O YOLO recebe e processa a imagem a ser analisada.

Camadas Convolucionais

- Aqui o YOLO fará uma série de padronizações, encontrando e rotulando as características da imagem - que não dependem de *input* manual -, tais como bordas, texturas, tamanho, largura, etc.

Camadas de *Pooling*

- ***Max pooling*** é utilizado para reduzir as dimensões espaciais da imagem enquanto mantém as características principais. É importante para que se mantenha a eficiência do modelo ao trabalhar com dimensões menores, porém, sem perda de informações.

Camadas Totalmente Conectadas

- Nessa parte as características obtidas nas camadas convolucionais serão interpretadas, mapeando as características da imagem para que as *bounding boxes* sejam montadas e relacionadas com as classes.

Grid da Imagem

- Aqui o modelo divide a imagem da entrada num grid (uma grade) de tamanho quadrado N. O modelo então passa pela imagem verificando:
 - Se há um objeto presente dentro daquela célula.
 - A classe do objeto (se houver um objeto).
 - Desenhar a *bounding box* do objeto

Bounding Boxes e Classes

- Cada *bounding box* prevê:
 - As coordenadas (x, y) do centro da caixa.
 - A largura e altura da caixa (w, h).
 - Um valor que representa a probabilidade do objeto, indicando a certeza da presença de um objeto dentro da caixa e a precisão da caixa delimitadora.
 - Um valor associado a cada classe de objeto que representa a confiança de que há um objeto daquela classe no interior da bounding box.
- Para cada célula, o YOLO prevê as classes de objetos, utilizando uma função de ativação ***softmax*** para calcular as probabilidades de cada classe, sendo esses valores de probabilidade os ***scores***.

Saída (Output)

- A saída final da rede YOLO consiste em uma **matriz** que contém diversas *bounding boxes*, cada uma contendo os elementos acima citados
- Após todo esse processo, temos o pós-processamento que envolve a aplicação da função *Non-Max Suppression*, ou NMS, que vai utilizar do *threshold* de probabilidade para eliminar caixas com probabilidades muito baixas e, por fim, aplicar o IoU (*Intersection over Union*), eliminando as bounding boxes redundantes que são geradas no mesmo possível objeto. Semelhante a como foi apresentado a Arquitetura do YOLO:

1. Thresholds (limite): Com todos os valores de saídas em mãos, para melhorar a eficiência da detecção, são definidos dois *thresholds*:

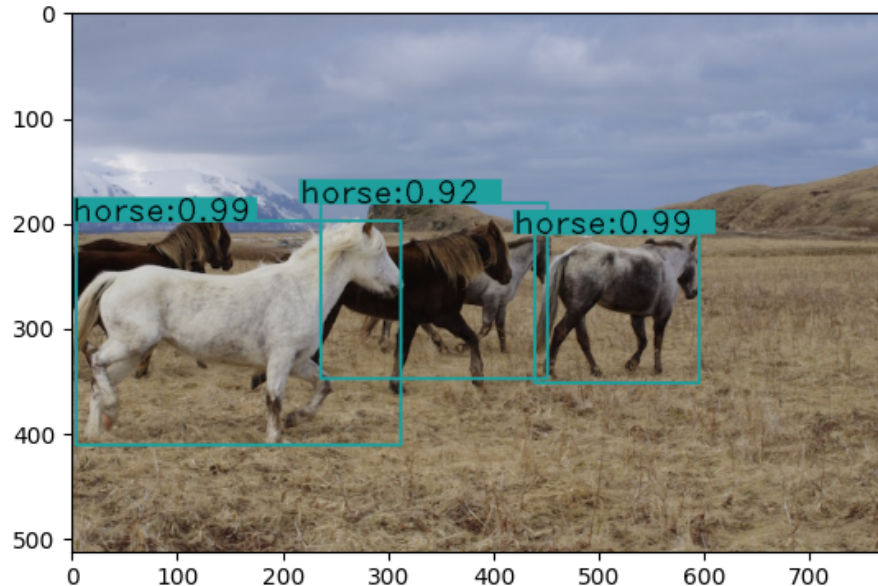
- um limiar de confiança para a eliminação de bounding boxes com valores muito baixos.
- um limiar para caixas se sobrepondo, para eliminar múltiplas caixas de um mesmo objeto

2. Non-Max Suppression (NMS): Esta é a função que irá realizar de fato o filtro. Primeiramente irá eliminar todas as *bounding boxes* que estão abaixo do limiar de probabilidade. Após o limiar de confiança ter sido aplicado, o critério de IoU é utilizado. As *bounding boxes* de um suposto mesmo objeto são comparadas e se tiverem uma intersecção acima do limiar pré-estabelecido, confirmando que de fato são do mesmo objeto, a que tiver o menor *score* será descartada.

Resultados

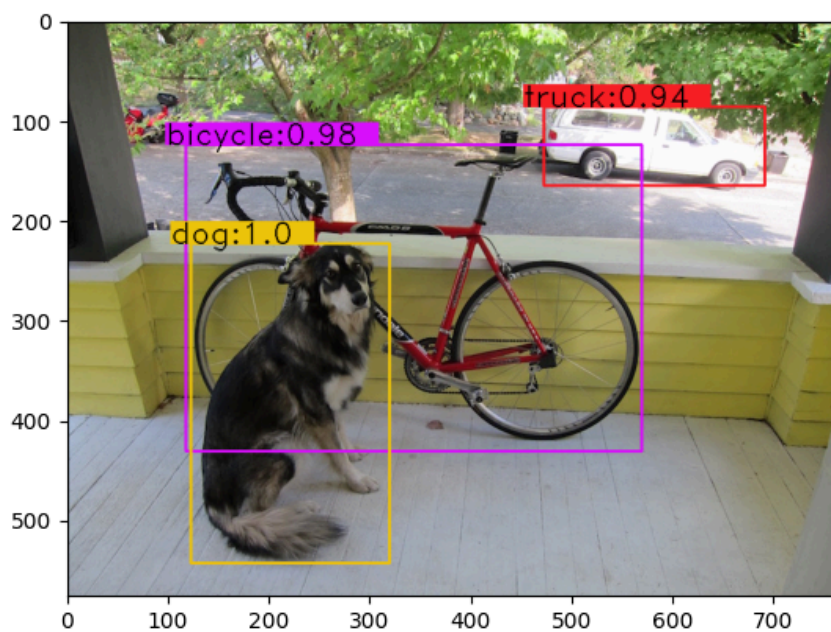
Tendo em vista o modelo YOLO observado, sua base teórica, motivação para implementação e escolha, finalmente alguns resultados serão apresentados. Nas próximas imagens é possível ver como o algoritmo funciona e notar seu grau de eficiência.

Figura 3 - Objeto de Classe única



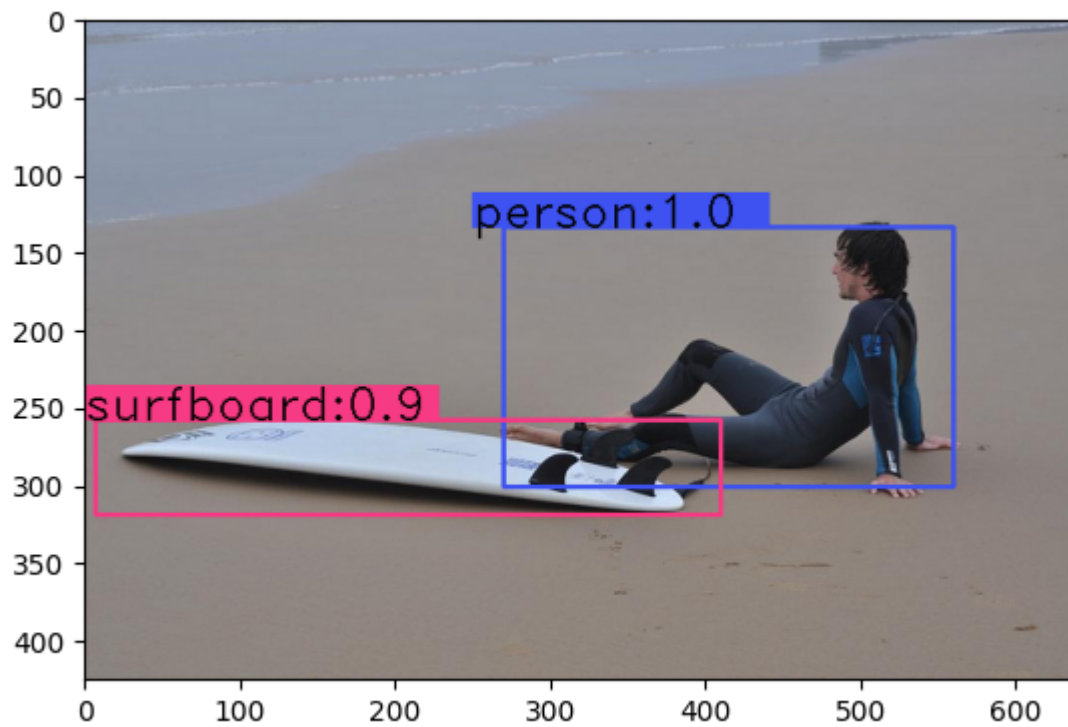
Fonte: Dataset do professor.

Figura 4 - Objetos de Classes variadas



Fonte: Dataset do professor.

Figura 5 - Dois Objetos de Classes diferentes



Fonte: Dataset do professor.

Figura 6 - Objetos e Classes variadas



Fonte: Google.

Como as figuras de 3 até 6 mostram, temos resultados bem acurados com relação às detecções dos objetos nas imagens. Os exemplos foram dispostos do mais simples, uma única classe de objeto, para o mais complexo, no qual temos vários objetos e de classes diferentes, além de graus de sobreposição relevantes. O valor do limiar para a existência do objeto foi 0.8, enquanto que o limiar do IoU foi de 0.5.

Figura 7 - Objetos e Classes com alta sobreposição



Fonte: Google.

Porém, apesar desses ótimos resultados, algumas dificuldades foram encontradas com outras imagens. Na figura 6 podemos notar vários objetos de classes diferentes todos sobrepostos, com pessoas no primeiro plano e um sofá no segundo plano. Como as pessoas bloqueiam a visão sobre o sofá, há menos informações visíveis sobre o objeto em segundo plano, assim, o algoritmo não possui tanta certeza sobre o objeto ou qual caixa deveria delimitá-lo, tanto que, é possível notar que a *bounding box* sobre o sofá não está posicionada adequadamente. Este é um exemplo em que diminuir o limiar de confiança é benéfico e necessário para o resultado, apesar de bons, os valores são consideravelmente mais baixos que nos outros exemplos.

Deste exemplo vemos que diminuir o limiar de confiança pode ser benéfico e até necessário para o obtermos o resultado correto; pois embora o cérebro humano consiga extrapolar as informações visíveis, computacionalmente, faltam dados para uma confiabilidade maior. Desta forma, concluímos que os valores de *Thresholds* e *NMS* não necessariamente devem ser absolutamente próximos de 1 para nos fornecer análises interessantes sobre a natureza da imagem. Para cada problema devemos considerar o quão puro são os dados que tentamos representar e analisar e quantos subgrupos podem estar contidos dentro da nossa expectativa da média.

Referências Bibliográficas

1. **LAPIX UFSC.** *Deep learning e detecção de objetos*. Disponível em: <https://lapix.ufsc.br/ensino/visao/visao-computacionaldeep-learning/deteccao-de-objetos-em-imagens/>.
2. MARIM, Yan Victor Ribeiro. *Detecção de objetos via CNN*. 2020. Projeto de Graduação (Bacharelado em Engenharia Elétrica) – Universidade Federal do Espírito Santo, Vitória, 2020. Disponível em: https://ele.ufes.br/sites/engenhariaeletrica.ufes.br/files/field/anexo/projeto_de_graduacao_ii_-_yan_victor_ribeiro_marim.pdf.
3. **Ultralytics Documentation.** *YOLO Models Library*. 2023. Disponível em: <https://docs.ultralytics.com/>.
4. REDMON, Joseph; DIVVALA, Santosh; GIRSHICK, Ross; FARHADI, Ali. *You Only Look Once: Unified, Real-Time Object Detection*. arXiv preprint, arXiv:1506.02640, 2015. Disponível em: <https://arxiv.org/abs/1506.02640>.
5. **Ultralytics Documentation.** *YOLO Performance Metrics*. Disponível em: <https://docs.ultralytics.com/guides/yolo-performance-metrics/#results-storage>. **V7 Labs.** *YOLO Object Detection Explained*. Disponível em: <https://www.v7labs.com/blog/yolo-object-detection>.
6. **GitHub - YOLOv3 from OpenCV Object Detection.** Disponível em: <https://github.com/computervisioneng/yolov3-from-opencv-object-detection>.